Kleisli Composition

eason: 7 · Advanced FP · Category: Functional · Level: Advanced · Build with Waq

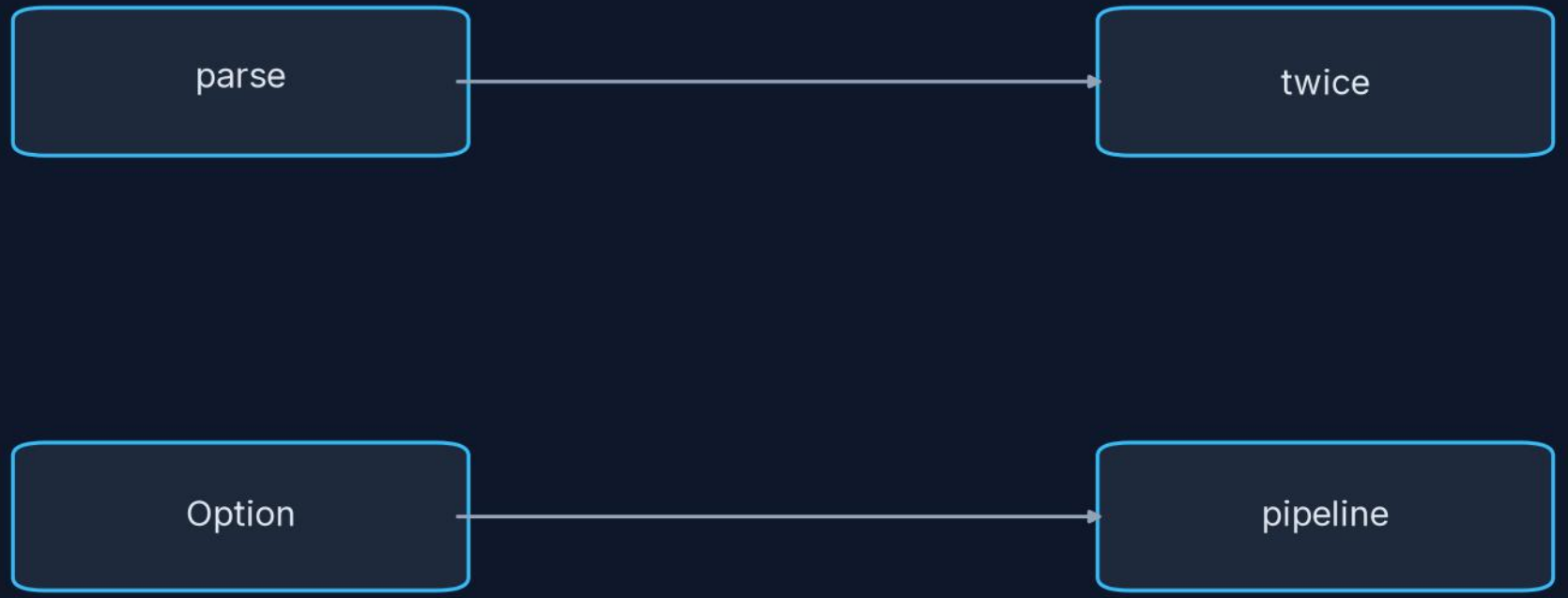
Today's Hook

- Today: Kleisli Composition as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Kleisli Composition structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Kleisli Composition
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/functional/kleisli/Kleisli.java
package com.buildwithwaqar.patterns.functional.kleisli;

import com.buildwithwaqar.patterns.functional.typeclass.Kind;
import com.buildwithwaqar.patterns.functional.typeclass.Monad;

import java.util.function.Function;

/** Kleisli arrow A =&gt; F&lt;B&gt; — composable effectful functions. */
public record Kleisli<F, A, B>(Function<A, Kind<F, B>> run) {
    public Kind<F, B> apply(A a) { return run.apply(a); }

    public <C> Kleisli<F, A, C> andThen(Monad<F> M, Kleisli<F, B, C> next) {
        return new Kleisli<>(a -> M.flatMap(run.apply(a), next.run));
    }

    public static <F, A, B> Kleisli<F, A, B> of(Function<A, Kind<F, B>> f) {
        return new Kleisli<>(f);
    }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.functional.kleisli;
import com.buildwithwaqar.patterns.functional.typeclass.OptionKind;
import dev.zazr.control.Option;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Kleisli walkthrough ===");
    Kleisli<OptionKind.Witness, String, Integer> parse = Kleisli.of(s -> {
      try { return OptionKind.lift(Option.some(Integer.parseInt(s))); }
      catch (Exception e) { return OptionKind.lift(Option.none()); }
    });
    Kleisli<OptionKind.Witness, Integer, Integer> twice = Kleisli.of(n -> OptionKind.lift(Option.some(n * 2)));
    var pipeline = parse.andThen(OptionKind.MONAD, twice);
    System.out.println("ok=" + OptionKind.lower(pipeline.apply("21")));
    System.out.println("bad=" + OptionKind.lower(pipeline.apply("x")));
  }
}
```

Walkthrough Output

```
=== Kleisli walkthrough ===
ok=Some(42)
bad=None
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Kleisli Composition
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Kleisli Composition → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.